SC2002 Group Assignment

Restaurant Rush

A turn-based restaurant management game in Java: design report

Item	Value
Group	<LAB GROUP> / Group <N>
Members	<NAME 1, NAME 2, NAME 3, NAME 4, NAME 5>
Repository	<PASTE GITHUB REPOSITORY URL>
Stage tags	`stage-1` (706c320) `stage-2` (beb2923) `stage-3` (027c290)
Video	<LINK, or "included in the zip">

> Text in **<angle brackets>** is for the group to fill in before submitting (names, links, reflections). Everything else was generated from the code and the actual test runs.

1. Overview

Restaurant Rush is a command-line, turn-based game. The player is the restaurant manager with one Waiter, one Chef and one Cashier (and, in Stage 3, a Host). Each staff member can do one task per turn, so the manager has to decide whom to seat, whose order to take, what to serve and what the Chef cooks first. The game runs exactly 18 turns. The player wins if at least 4 customers pay, revenue is at least $45.00 and the average satisfaction of paid customers is at least 60.

The game was built in three stages. Each stage is a tagged, runnable version in GitHub, and each builds on the one before.

Stage	Adds	Tag / commit	Result
1	Regular customers, menu, Waiter, Chef, Cashier, Table, Order, waiting queue, the six-step turn, abandonment, payment, victory and defeat	`stage-1` 706c320	Runs on its own; victory and defeat runs reproducible
2	VIP and Critic customers (different patience, bill and ordering), dispatched through the customer objects	`stage-2` beb2923	Runs on its own; Stage 1 replays unchanged
3	Host, ComboMeal, Happy Hour	`stage-3` 027c290	Runs on its own; Stage 1 and 2 replay word for word

How to run. `scripts/build.sh` compiles with plain `javac` (JDK 17 or newer); `scripts/run.sh` plays the Stage 3 game, `scripts/run.sh --classic` the Stage 2 base setting and `scripts/run.sh --regular-only` the Stage 1 setting. `mvn test` runs the 68 unit tests. Every recorded run can be replayed with `scripts/replay.sh <script>`.

Architecture. The code follows Boundary / Control / Entity. `ConsoleUI` (boundary) only reads choices and prints state. `GameController` (control) runs the turn sequence and the victory check. All game rules live in entity classes such as `Customer`, `Order`, `Waiter` and `HappyHour`. Main source: 30 files, about 1,900 lines; tests: about 1,400 lines.

2. UML class diagram (final design, matches stage-3)

Visibility: + public, - private, # protected, ~ package-private. Multiplicities are shown on the associations. Annotations for encapsulation, polymorphism, composition and inheritance are on the diagram. A script (`scripts/check_diagram.py`) checks that every class, attribute and operation in the diagram exists in the code with the same number of parameters, and that every type in the code appears in the diagram.

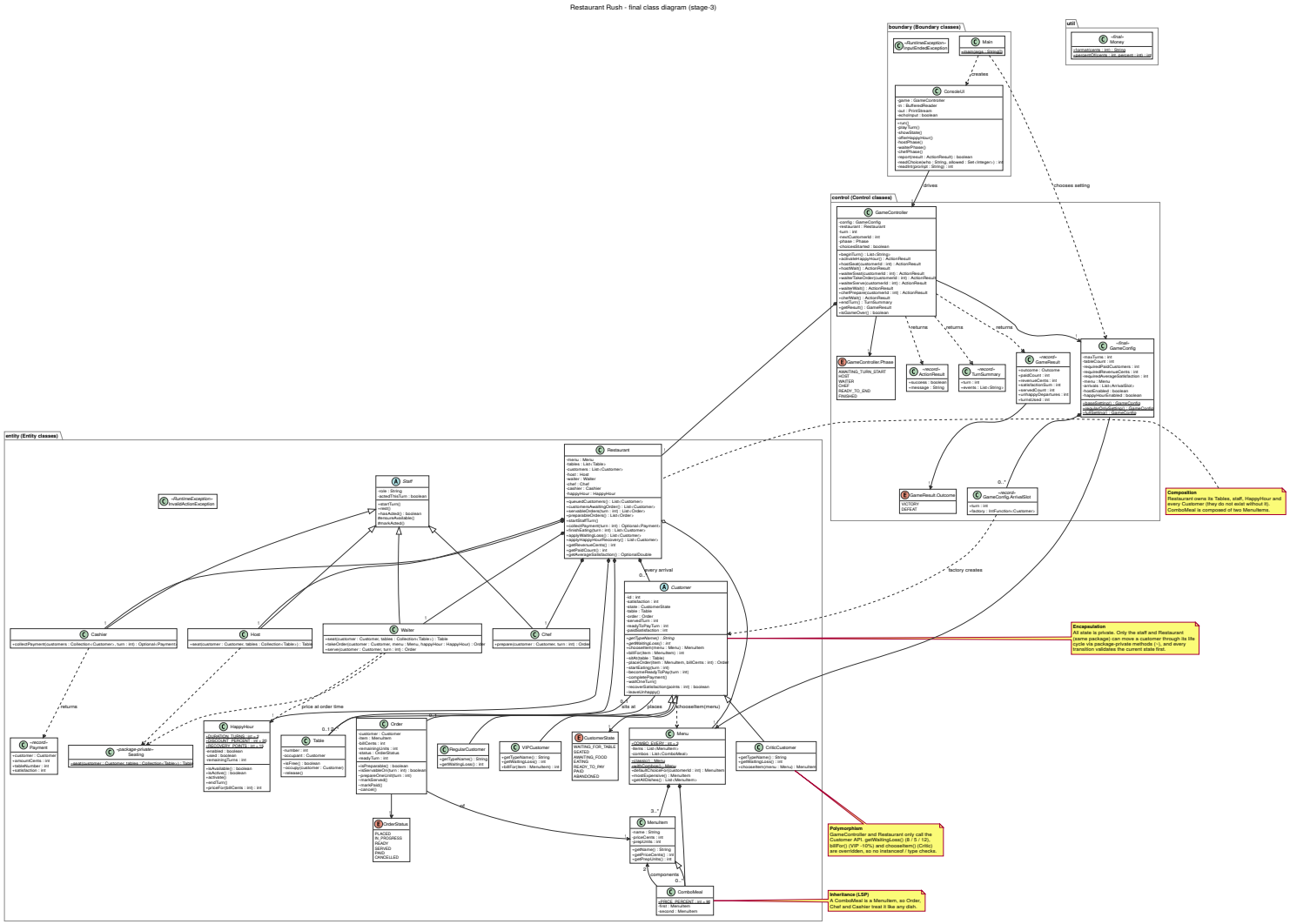


Figure 1. Complete class diagram (boundary, control, entity, util). Vector graphic: zoom in the PDF to read the members. Also in docs/diagrams/class-diagram.svg

2.1 Focused views of the class diagram

The views below are generated from the same class blocks as Figure 1, so they are consistent with it, and are easier to read.

View A: restaurant, staff and Happy Hour

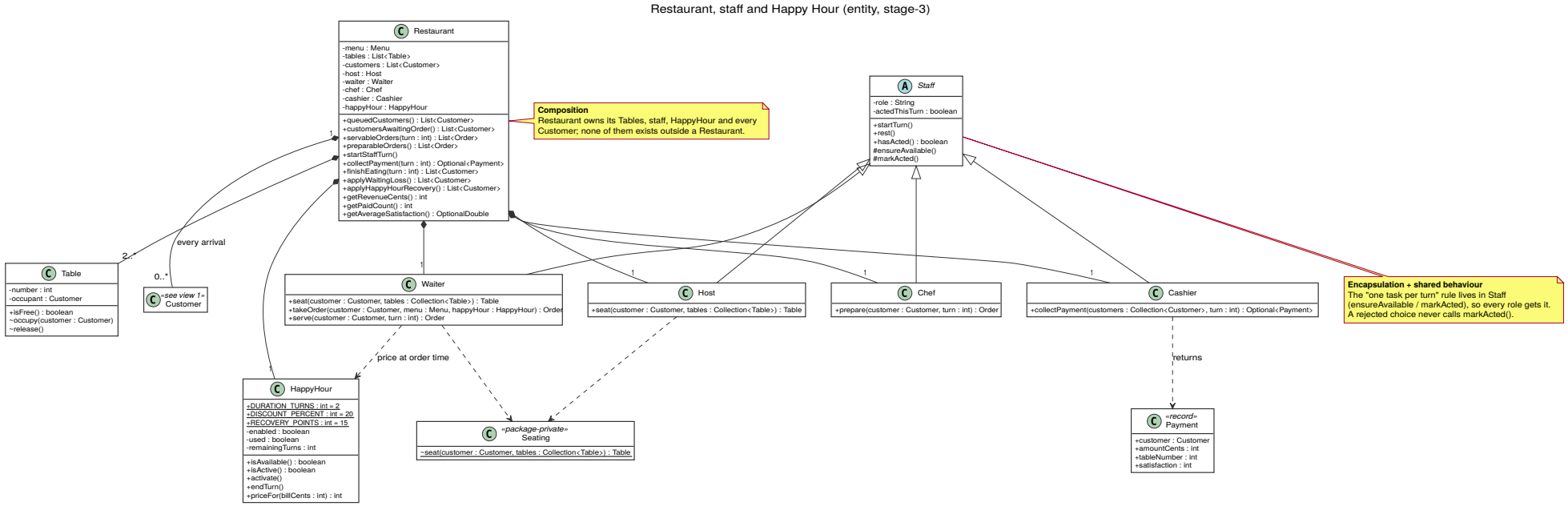


Figure 2. The Restaurant aggregate, the staff hierarchy and the pricing policy.

- **Composition.** `Restaurant` owns its tables, its four staff members, its `HappyHour` policy and every customer who arrives. They have no meaning outside it. `ComboMeal` is composed of two `MenuItems`.
- **Shared behaviour in the base class.** The rule "one task per turn" is implemented once in `Staff` (`ensureAvailable`, `markActed`). `markActed` is only reached after an action has succeeded, so a rejected choice never uses up the task.
- **Seating** is a rule shared by `Waiter` and `Host`; it lives in the package-private `Seating` so the two roles cannot drift apart.

View B: customers, orders and the menu

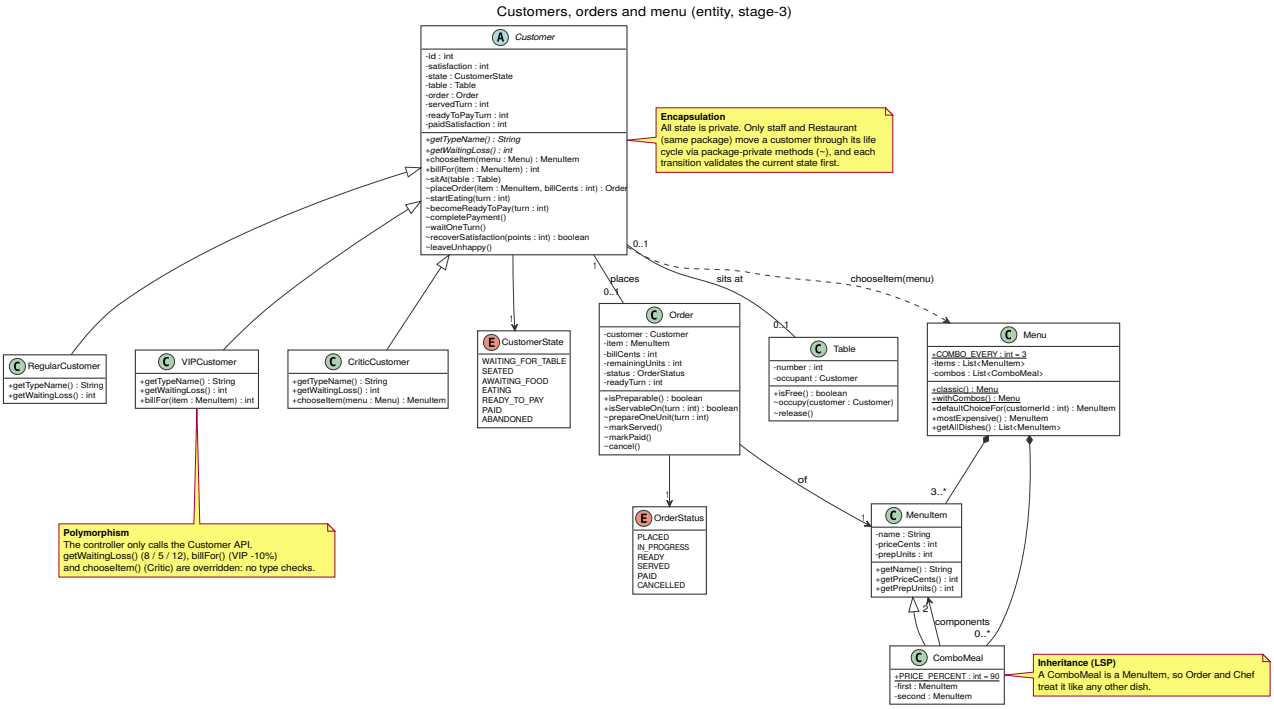


Figure 3. Customer hierarchy, order life cycle and menu (including ComboMeal).

- **Encapsulation.** All `Customer` state is private. Only classes in the entity package (the staff and `Restaurant`) can move a customer through its life cycle, through package-private methods such as `sitAt`, `placeOrder`, `startEating` and `completePayment`, and each one first checks the current state. The controller and the UI cannot put a customer into an impossible state.
- **Polymorphism.** `getWaitingLoss()` (8 / 5 / 12), `billFor()` (VIP 10% off) and `chooseItem()` (Critic orders the most expensive dish) are overridden in the subclasses. `GameController` and `Restaurant` only call the `Customer` API, so the turn logic has no `instanceof` or type checks (a test checks this).
- **Inheritance and substitution.** `ComboMeal` is a `MenuItem` (price and units computed from its two components), so `Order`, `Chef` and the `Cashier` handle a combo exactly like a single dish.

View C: boundary and control

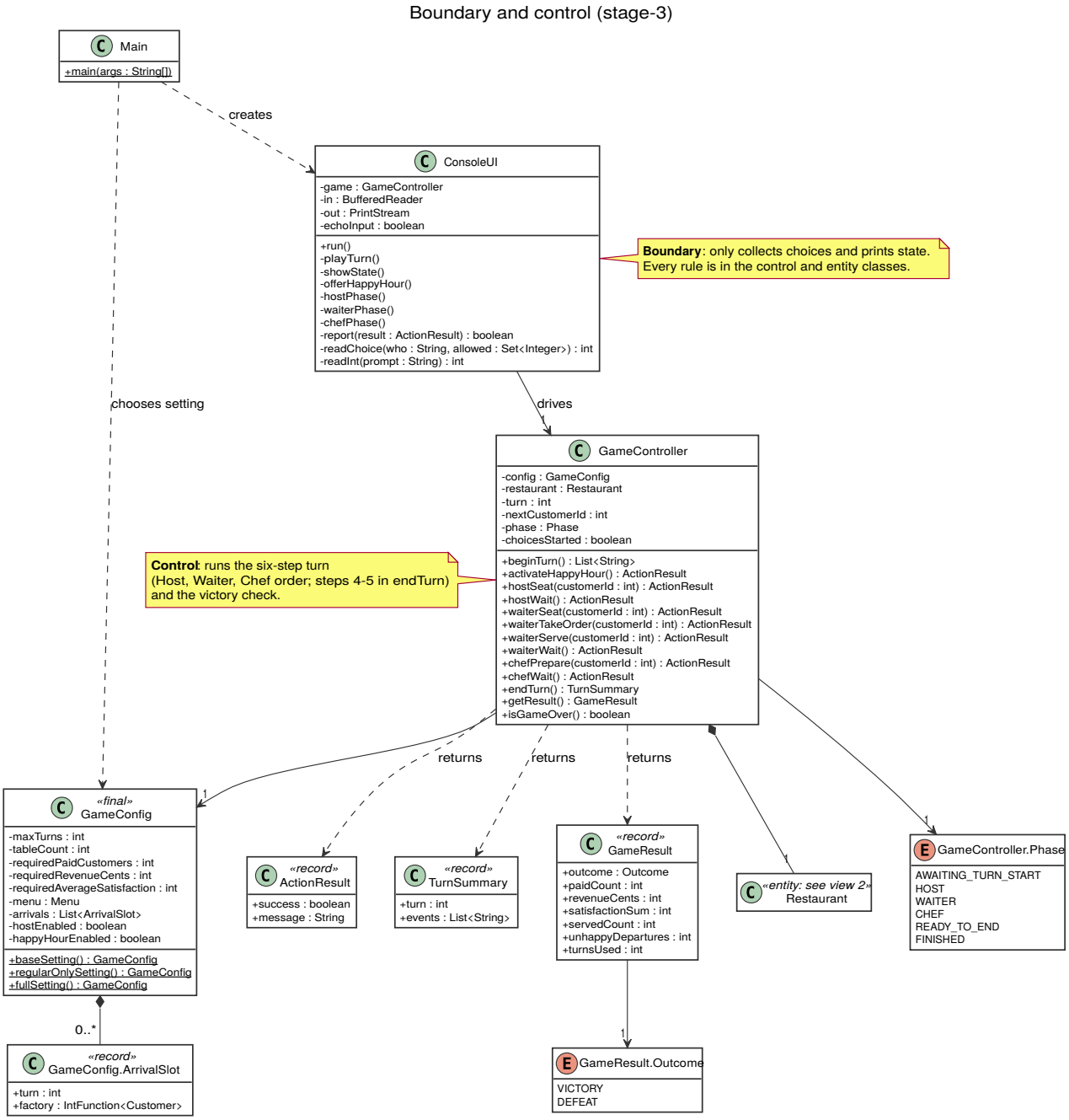


Figure 4. The boundary class talks to one control class; control returns small result records.

- `ConsoleUI` has no rules. It asks `GameController` to do something and prints the `ActionResult`. A failed choice is explained and the same question is asked again.
- `GameController` enforces the order of a turn with an explicit `Phase` (Host, Waiter, Chef, end of turn) and decides the result after turn 18.
- `GameConfig` holds every number of a game and is the only place that names concrete customer classes (as `IntFunction<Customer>` factories).

3. UML sequence diagram

Scenario. The VIP (customer 2) arrives on turn 4. The manager switches Happy Hour on, the Host seats the VIP, the Waiter takes the order, the Chef cooks it, the dish becomes READY, is SERVED on turn 5, the VIP eats during turn 5 and finishes on turn 6, and the Cashier collects the payment on turn 7. After turn 18 the result is checked. The diagram includes one invalid manager choice (guard and loop), the Happy Hour guard, and the VIP discount followed by the Happy Hour discount.

How the manager's choice changes the outcome. If the manager answers "No" to Happy Hour, the VIP orders the same Burger but the bill is locked at $13.50 and the VIP pays with satisfaction 95. With Happy Hour the VIP pays $10.80 ($15.00 x 0.9 x 0.8) with satisfaction 100. The same run played both ways ends with revenue $91.50 without Happy Hour and $88.80 with it, and total satisfaction 399 against 404 (average 79.8 against 80.8).

The full diagram is split into four parts so it stays readable (the unsplit version is docs/diagrams/sequence-diagram.svg).

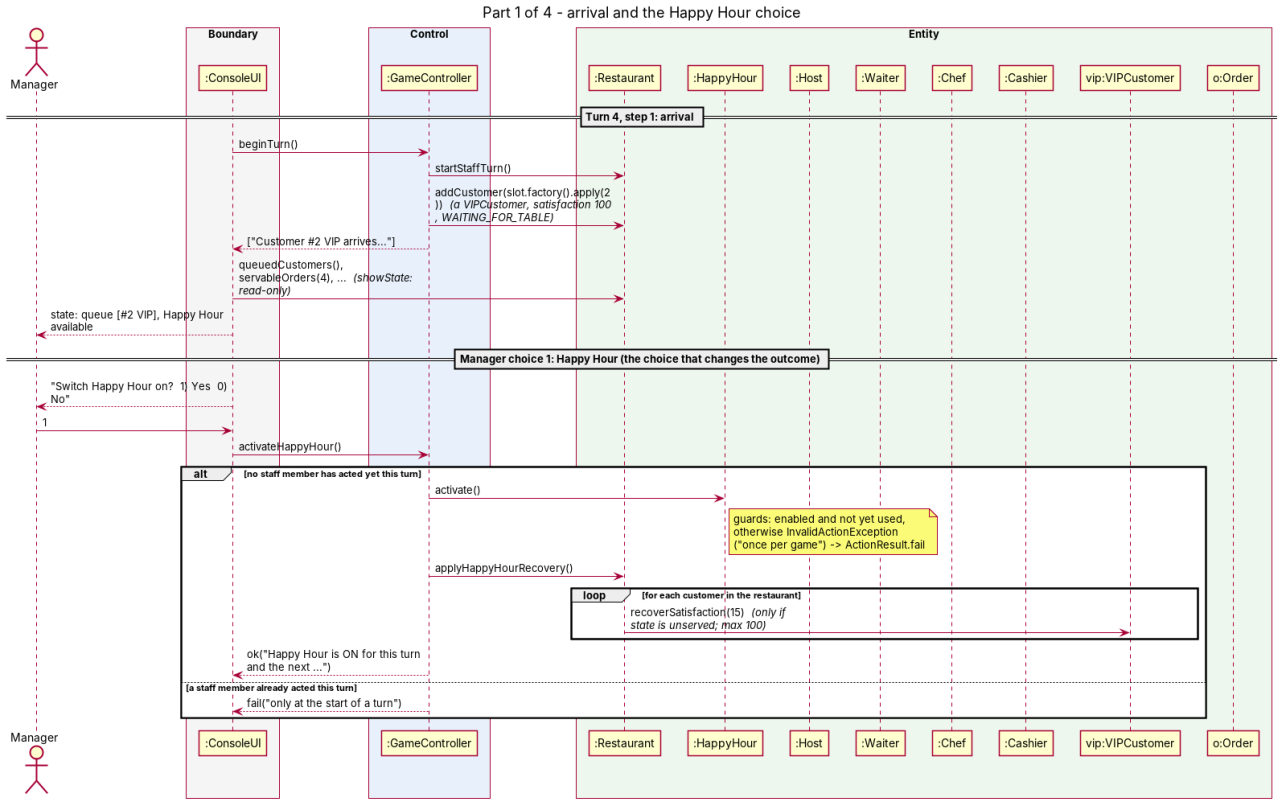


Figure 5. Part 1: arrival (step 1) and the Happy Hour choice, with its guard.

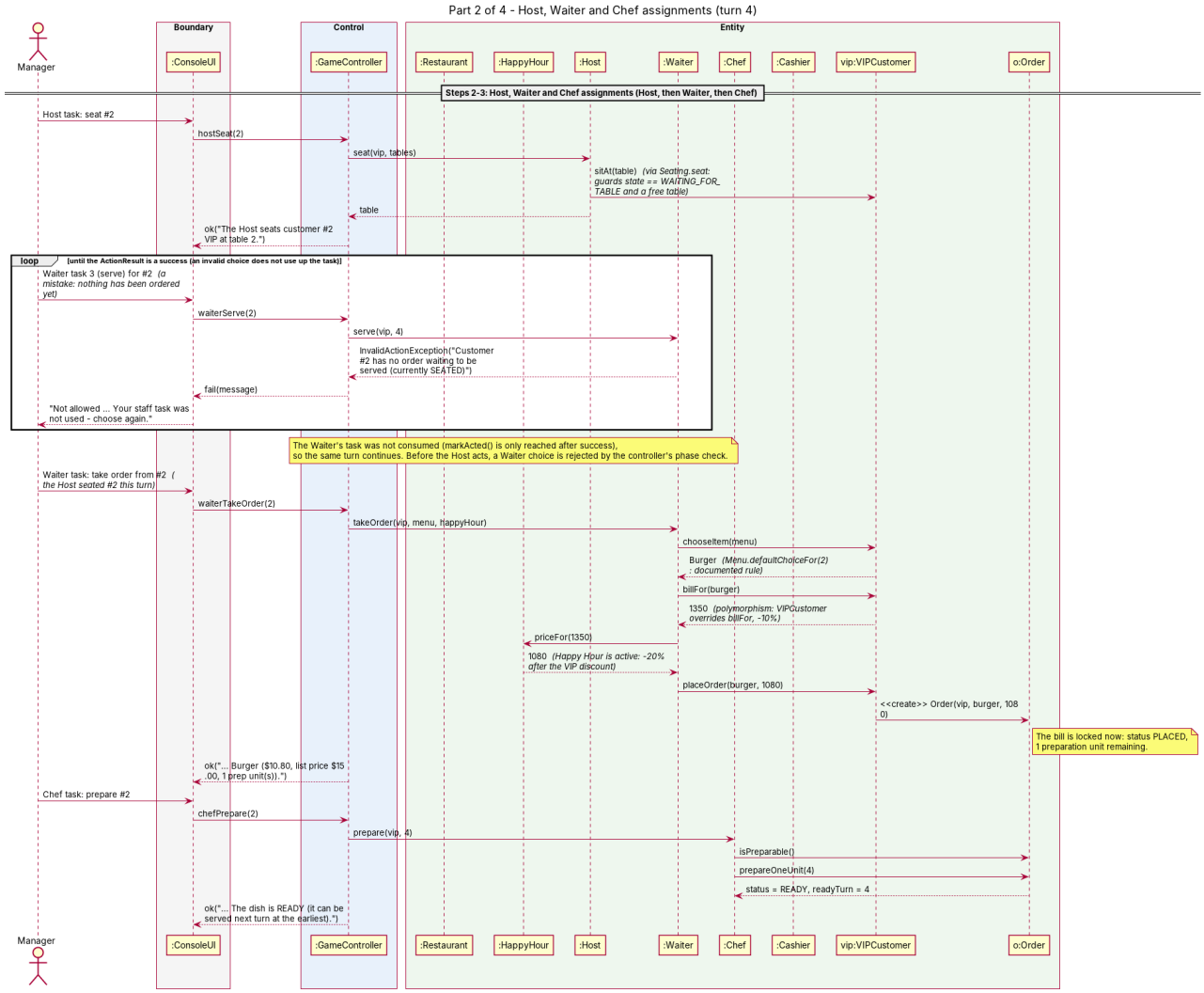


Figure 6. Part 2: Host, Waiter and Chef assignments (steps 2 and 3). The invalid choice is rejected without using the task, and the loop asks again.

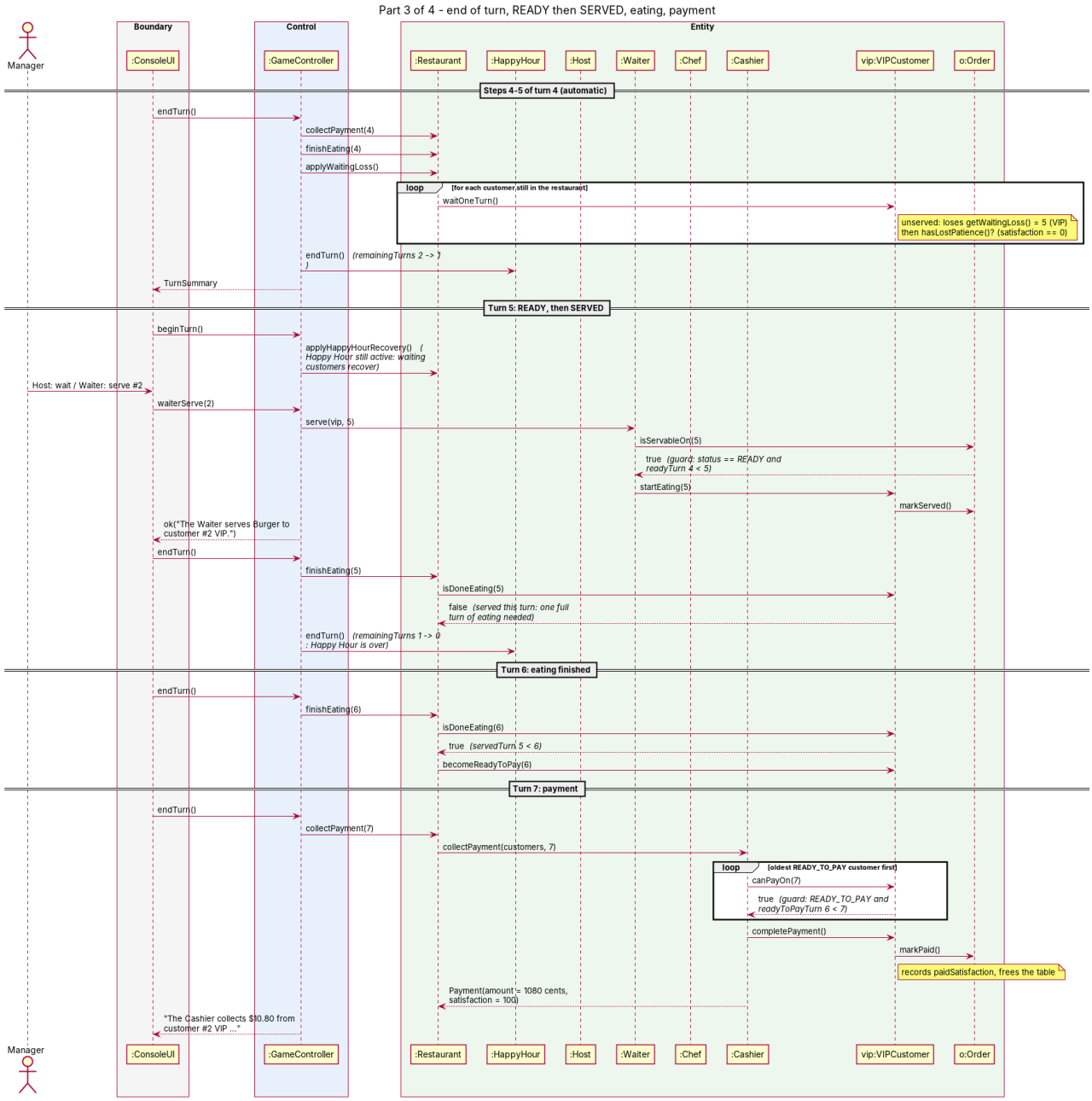


Figure 7. Part 3: end of turn 4, READY then SERVED on turn 5, eating, and payment on turn 7, with the guards on the one-turn delays.

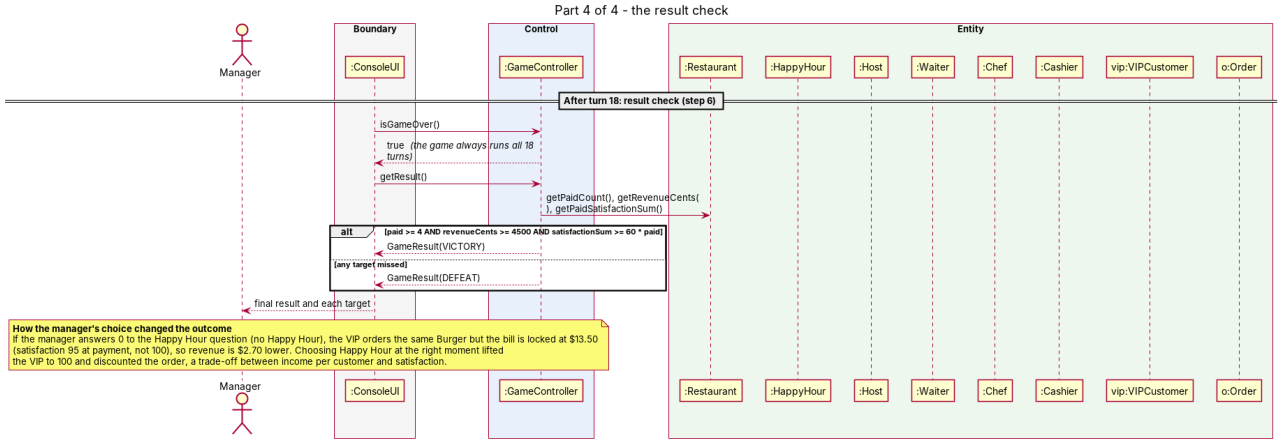


Figure 8. Part 4: the result check after turn 18 and how the manager's choice changed the outcome.

4. Design discussion

4.1 Key abstractions and their responsibilities

Class	Responsibility (what it knows and does)
`Customer` (abstract), `RegularCustomer`, `VIPCustomer`, `CriticCustomer`	Own satisfaction and life cycle (queue, seated, awaiting food, eating, ready to pay, paid, abandoned). Define how fast patience runs out, how the bill is calculated and what is ordered. Subclasses override only those three things.
`Order`, `OrderStatus`	One customer's order: dish, locked bill, remaining preparation units and status. Knows when it can be served (`isServableOn`).
`MenuItem`, `ComboMeal`, `Menu`	Dishes with price in cents and preparation units. `ComboMeal` composes two dishes. `Menu` holds the dishes and the documented rule for what a customer orders.
`Table`	Occupied until the customer pays or leaves.
`Staff` (abstract), `Waiter`, `Host`, `Chef`, `Cashier`	Each performs the tasks of one role and validates them. `Staff` enforces one task per turn.
`Restaurant`	Aggregate of tables, staff, customers and Happy Hour. Answers "who can be seated / served?", runs the automatic end-of-turn steps, and derives all statistics (revenue, paid count, average satisfaction) from customer state.
`HappyHour`	Availability (once per game), duration (two turns), 20% price rule and the recovery amount.
`GameController`, `GameConfig`	Control: order of the six steps and phases, result after turn 18; the numbers of a game and the arrival schedule.
`ConsoleUI`	Boundary: reads and validates the format of input, prints state and messages. No rules.
`ActionResult`, `TurnSummary`, `GameResult`	Small immutable records returned by the controller.
`Money`	Integer-cent formatting and half-up percentage rounding, so no floating-point money.

4.2 Design principles (only where the code really shows them)

Principle	Where it is reflected	Honest limits
SRP	`ConsoleUI` only does I/O. `GameController` only sequences a turn. `Customer` owns its life cycle, `HappyHour` owns its numbers, `Seating` owns the seating rule, `Money` owns rounding.	`Restaurant` has several jobs (queries, automatic steps, statistics); we kept them together because they all need the same collections.
OCP	Stage 1 to 2 changed 6 main files (+96 / -5 lines). The turn logic in `GameController` was not touched (4 added lines in a message string). A new customer kind is a new subclass plus one line in `GameConfig`. `ComboMeal` needed no change in `Order`, `Chef` or `Cashier`.	Stage 3 did modify earlier classes: `Waiter.takeOrder` gained a parameter, `Menu.defaultChoiceFor` gained the combo rule, and the controller gained a Host phase. These were justified in the README.
LSP	Every `Customer` subclass can replace another in the same flow (test: all three kinds go from seating to payment through the same calls). A `ComboMeal` can be used wherever a `MenuItem` is. Every `Staff` subclass honours the one-task rule.	Subclasses change only numbers and choices, never the preconditions of the shared methods.
ISP	We deliberately created no interfaces (the brief says not to create them for a checklist). The customer's life-cycle methods are package-private, so other layers see only a small public API.	Weak: `ConsoleUI` sees the whole `GameController` API, including Host methods when there is no Host.
DIP	The controller depends on the `Customer` abstraction: `GameConfig` supplies customers as `IntFunction<Customer>` and is the only place that names the concrete classes. Orders and staff depend on `MenuItem`, not on `ComboMeal`.	`ConsoleUI` depends on the concrete `GameController`, and `Restaurant` creates its staff directly. Acceptable with a single implementation of each.

4.3 Trade-offs

Decision	Benefit	Cost
Rule violations throw `InvalidActionException`; the controller turns it into an `ActionResult`	The state is untouched when a choice fails, one message path, and the staff task is only marked used after success	Exceptions used for expected input errors
Statistics derived from customer state, no counters	Revenue cannot include an unpaid or abandoned order; nothing to keep in sync	Recomputed on every call (trivial at 6 customers)
Package-private life-cycle methods	Real encapsulation of customer and order state	Entity rule tests must be in the same package; staff classes must live in the entity package
Host, combos and Happy Hour are switches in `GameConfig`	Stage 1 and 2 runs replay word for word on the Stage 3 code	A few conditionals on `hostEnabled` in the controller
Explicit `Phase` in the controller	The order Host, Waiter, Chef is enforced and testable	One more piece of state to keep correct
A choice takes effect as soon as it is made (not queued until step 3)	The Chef's list already contains the order the Waiter just took, as the brief requires	Step 3 is not a separate method; the `readyTurn` guard makes the one-turn delay explicit
Scripted input echoed to the output	Reproducible, reviewable runs and transcripts	Prompts must stay stable (scripts are per stage)

4.4 An alternative design we considered

Alternative: one `Customer` class with a `CustomerType` enum and a `switch` in the turn controller. Satisfaction loss, discount and food choice would be `switch (customer.type)` blocks inside `GameController`. It is shorter for Stage 2 (no new classes), but every new kind of customer, and every new rule that depends on the kind, would edit the controller; the brief also forbids this for Stage 2. We rejected it. With the subclass design, Stage 2 added two small classes and did not touch the turn logic.

We also considered a **Command pattern** for staff actions (`SeatCommand`, `ServeCommand`, ...). It would give uniform logging and undo, but for six actions it adds many classes without a requirement for them. Methods on `Staff` subclasses plus controller wrappers were enough.

4.5 How the player's choices change the result

Choice	Evidence (same crowd, base setting)	Effect
Act or do nothing	Passive manager (always Wait) vs. the greedy manager (serve first, then order, then seat; Chef takes the shortest job)	Passive: DEFEAT, 0 paid, $0.00, 2 unhappy departures. Greedy: VICTORY, 5 paid, $91.50 (Stage 3, no Happy Hour).
Switch Happy Hour on when the VIP arrives	Stage 3 run with and without Happy Hour	VIP bill $10.80 instead of $13.50 and satisfaction 100 instead of 95. Revenue $88.80 vs $91.50; average satisfaction 80.8 vs 79.8.
Whom the Waiter helps first	Every turn the Waiter can do only one of: seat, take an order, serve	A dish that is READY waits for the Waiter while customers keep losing 8 / 5 / 12 points per turn, so the order of tasks decides who reaches 0 and leaves unpaid.
Using the Host	Host seats before the Waiter acts	A customer seated by the Host can have the order taken the same turn, and the dish can be served one turn earlier.

5. Functional evidence

5.1 Automated tests (68, all passing)

Test class	Tests	What it proves
`TurnRulesTest`	15	The brief's worked example turn by turn; Waiter before Chef; one task per turn; READY then served next turn; invalid choices keep the task; abandonment cancels the order and frees the table; revenue excludes unpaid orders; targets are inclusive; 18 turns always; at most five can pay
`EntityRulesTest`	10	Food rule; Cashier one payment per turn, oldest first, never on the turn they became ready; payment frees the table; served customers lose nothing; satisfaction never below 0
`CustomerBehaviourTest` (stage 2)	8	Waiting loss 8 / 5 / 12; VIP 10% off; Critic orders the most expensive dish; all kinds follow the same flow; the controller has no type checks
`StageThreeEntityTest` (stage 3)	11	Combo price, units and rounding; who orders a combo; VIP then Happy Hour = $10.80; Happy Hour duration, once only, recovery capped at 100; Host and Waiter share seating rules
`StageThreeFeaturesTest` (stage 3)	14	Host acts first and in addition to the Waiter; Happy Hour only at turn start; remaining duration 2 then 1; recovery on both turns; price locked at order time; full setting winnable
`ScriptedRunTest`	8	All six victory and defeat scripts through the real UI; Stage 1 and 2 runs replay word for word on the Stage 3 code; bad input is rejected and asked again
`MoneyTest`	2	Integer cents formatting and half-up rounding

5.2 Reproducible runs (base setting)

Stage	Run	Command	Result
1	Victory	`scripts/replay.sh scripts/stage1-victory.txt --regular-only`	VICTORY: 5 paid, $66.00, average 79.2
1	Defeat	`scripts/replay.sh scripts/stage1-defeat.txt --regular-only`	DEFEAT: 0 paid, 2 unhappy departures
2	Victory	`scripts/replay.sh scripts/stage2-victory.txt --classic`	VICTORY: 5 paid, $73.50, average 75.6
2	Defeat	`scripts/replay.sh scripts/stage2-defeat.txt --classic`	DEFEAT: 0 paid, 2 unhappy departures
3	Victory	`scripts/replay.sh scripts/stage3-victory.txt`	VICTORY: 5 paid, $88.80, average 80.8 (Happy Hour used)
3	Defeat	`scripts/replay.sh scripts/stage3-defeat.txt`	DEFEAT: 0 paid, 2 unhappy departures

Full transcripts are in `docs/runs/`. The Stage 1 and 2 transcripts replay word for word on the Stage 3 code (checked by `ScriptedRunTest`), which is our evidence that earlier behaviour still works.

5.3 Excerpt: turn 4 of the Stage 3 victory run

```
========================== TURN 4 of 18 ==========================
Customer #2 VIP arrives and joins the waiting queue (satisfaction 100).
Happy Hour is available (once per game; this turn and the next: 20% off new orders ...).
Switch it on now?  1) Yes  0) No
Happy Hour choice> 1
  -> Happy Hour is ON for this turn and the next: new orders get 20% off. ...
Host choice> 1
Customer id to seat> 2
  -> The Host seats customer #2 VIP at table 2.
After the Host - Waiter can: seat [none] | take order [#2] | serve [none]
Waiter choice> 2
  -> The Waiter takes the order of customer #2 VIP: Burger ($10.80, list price $15.00, 1 prep
unit(s)).
Chef choice> 1
  -> The Chef prepares Burger for customer #2 VIP (unit 1/1). The dish is READY (it can be served
next turn at the earliest).
--- End of turn 4 ---
```

5.4 Where each rule of the brief is implemented and tested

Rule	Implemented in	Test
Base setting (18 turns, 2 tables, arrivals, targets)	`GameConfig.baseSetting()`	`baseSettingMatchesTheBrief`
Waiter acts before Chef	`GameController` phases	`waiterMustChooseBeforeTheChef`
A dish finished this turn is served next turn at the earliest	`Order.isServableOn`	`aDishFinishedThisTurnIsOnlyServedNextTurn`
Cashier: one payment per turn, oldest first, not on the turn ready	`Cashier.collectPayment`	`cashierPaysOneCustomerPerTurn...`
Unserved customers lose satisfaction; at 0 they leave	`Restaurant.applyWaitingLoss`	`customerLeavesAtZeroSatisfaction...`
Invalid choice explains why and does not use the task	`Staff.markActed` after success; `ActionResult`	`invalidChoiceExplainsWhy...`
Revenue never includes unpaid or abandoned orders	`Restaurant.getRevenueCents` (derived)	`customerLeavesAtZeroSatisfaction...`
The game always runs all 18 turns; victory targets	`GameController.getResult`	`victoryRequiresAllThreeTargets...`
At most five customers can pay	Turn arithmetic	`atMostFiveCustomersCanPay...`
VIP and Critic through the customer objects	`Customer` overrides	`theTurnControllerNeverChecksCustomerTypes`
Host seats one extra customer per turn	`Host`, `Seating`	`theHostSeatsOneCustomerIn...`
ComboMeal: units added, 90% price rounded	`ComboMeal`	`comboPriceIsNinetyPercent...`
Happy Hour: once, two turns, 20% off after VIP, +15 recovery, price locked	`HappyHour`, `Waiter.takeOrder`	`StageThreeFeaturesTest`
Money in integer cents	`Money`	`MoneyTest`
Rules out of the UI	`ConsoleUI` only calls the controller	`ScriptedRunTest` (whole game through the UI)

6. Thinking and development process

6.1 Stage snapshots

Tag	Commit	Main files changed	What the snapshot shows
`stage-1`	706c320	24 main + 7 test files (whole project)	A complete playable game with the Boundary/Control/Entity split, 30 unit and scripted tests, reproducible victory and defeat runs
`stage-2`	beb2923	6 files changed or added (+96 / -5 lines)	Two new customer classes; the turn logic untouched; Stage 1 transcripts identical
`stage-3`	027c290	12 files changed or added (+451 / -31 lines)	Host, ComboMeal, Happy Hour; seating rule extracted; earlier transcripts identical

6.2 Change analysis

The class diagram as it was at each earlier tag is in Appendix B (and `docs/diagrams/stages/`); the same script checks each one against the source of its own tag.

Stage 1 to 2. New: `VIPCustomer`, `CriticCustomer`. Changed: `Menu` (added `mostExpensive`), `GameConfig` (the base setting schedules a VIP and a Critic; `regularOnlySetting` keeps Stage 1), `GameController` (one message shows the list price when a discount applies; the turn logic is untouched), `Main` (a flag). Nothing was deleted. The change was small because Stage 1 already made `Customer` abstract with overridable patience, bill and food choice.

Stage 2 to 3. New: `Host`, `Seating`, `ComboMeal`, `HappyHour`. Changed: `Waiter` (seating delegated to `Seating`; `takeOrder` takes the Happy Hour so the bill is discounted once, when the order is taken), `Menu` (combos and their ordering rule), `Customer` (`recoverSatisfaction`), `Restaurant` (owns Host and Happy Hour), `GameController` (Host phase, Happy Hour), `GameConfig` (two switches), `ConsoleUI` and `Main`. Nothing was deleted. The six steps of a turn were not rewritten, and the saved Stage 1 and 2 transcripts still match, so earlier behaviour was not broken.

6.3 Team contribution and use of AI

Member	Main responsibility	Video segment
<Name 1>	<e.g. controller and turn loop>	<#>
<Name 2>	<e.g. customers and billing>	<#>
<Name 3>	<e.g. staff, tables, Host>	<#>
<Name 4>	<e.g. console UI, combos, Happy Hour>	<#>
<Name 5>	<e.g. tests, diagrams, report>	<#>

<Describe, honestly, how the group used AI assistance on this project (for example: code, tests, diagrams and this report were drafted with Claude and then reviewed, run and edited by the group). Each member also submits the course Declaration on Use of GAI Assistance.>

7. Reflections

7.1 Project-level lessons (draft: edit to match your experience)

- **Design for the next stage in the first stage.** Making `Customer` abstract with overridable patience, billing and food choice in Stage 1 meant Stage 2 did not touch the turn logic. A `CustomerType` switch would have needed controller edits for every new rule.
- **Where the design was not closed to change.** Stage 3 still needed edits to `Waiter.takeOrder`, `Menu.defaultChoiceFor` and the controller (a Host phase). Making features configuration switches limited the damage and let us prove with saved transcripts that earlier behaviour was unchanged.
- **Diagrams must be checked against the code.** The first draft of the sequence diagram showed the Waiter rejecting a choice made before the Host had acted, but in the code the controller's

phase check rejects it before the Waiter is called. We corrected the diagram, and added a script that checks the class diagram against the source.

- **Make a rule an object, then test it once.** Keeping "one task per turn" in `Staff` and the seating rule in `Seating` meant the Host got both for free and the tests were written once.
- **What we would do differently.** Introduce an interface between the UI and the controller if a second boundary (for example a GUI) were needed, and consider command objects for staff actions if undo or logging were required.

7.2 Individual reflections

<Each member: one short paragraph on the part you built or explained in the video, one thing you learned about object-oriented design from it, and one thing you would change.>

Member	Reflection
<Name 1>	<...>
<Name 2>	<...>
<Name 3>	<...>
<Name 4>	<...>
<Name 5>	<...>

Appendix A: how we read the brief

The full list is in `docs/GAME_NOTES.md`. The main decisions:

- Arrivals happen before the start-of-turn display, so the manager sees the new customer.
- Customer ids follow arrival order. What customers order is deterministic: customer n orders dish (n - 1) mod 3 (Salad, Burger, Pasta, ...). With combos on the menu, customers 3 and 6 order a combo instead.
- The manager chooses whom to seat; the customer sits at the lowest-numbered free table.
- Critic special behaviour (our definition): always orders the most expensive dish on the menu.
- "Oldest first" for the Cashier means longest in READY_TO_PAY (ties: lower id).
- A dish that is READY but not yet served still counts as "not served", so that customer keeps losing satisfaction.
- The Host acts before the Waiter, so a customer seated by the Host can order in the same turn.
- Happy Hour is offered at the very start of a turn while unused; switching it on at turn n covers turns n and n+1; the price is locked when the order is taken; the VIP discount is applied first, then Happy Hour.
- The average-satisfaction target is compared with integers (sum >= 60 x paid customers), so there is no rounding error.

Appendix B: the class diagram at stage-1 and stage-2

Derived from the final diagram by `scripts/stage_diagrams.py` and checked against the source at each tag. Compare with Figure 1 to see what each stage added: Stage 2 adds `VIPCustomer` and `CriticCustomer` (and `Menu.mostExpensive`); Stage 3 adds `Host`, `Seating`, `ComboMeal`, `HappyHour` and the Host phase, and `Waiter.takeOrder` gains the `HappyHour` parameter.

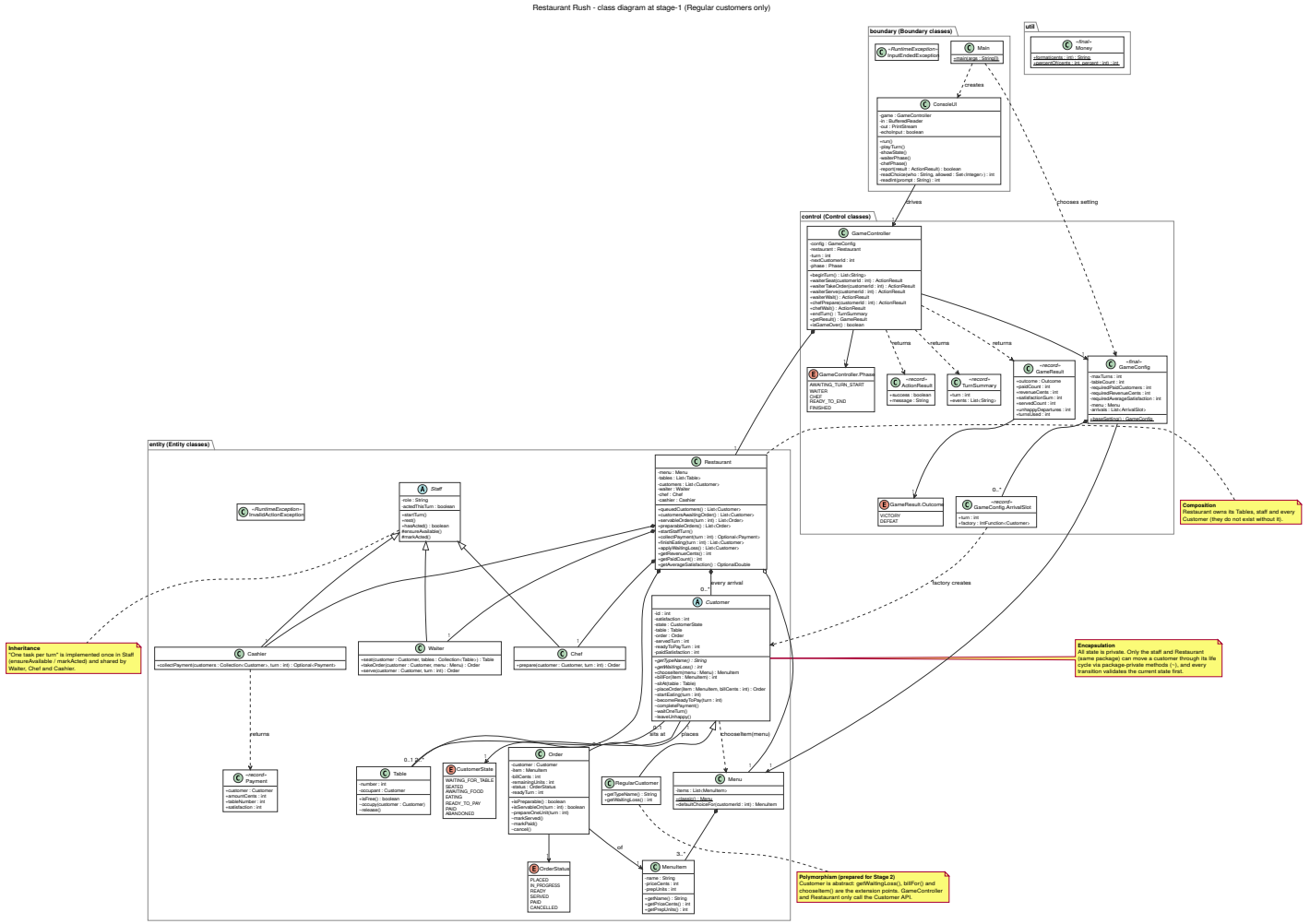


Figure B1. Class diagram at tag stage-1 (Regular customers only). Vector graphic: zoom to read.

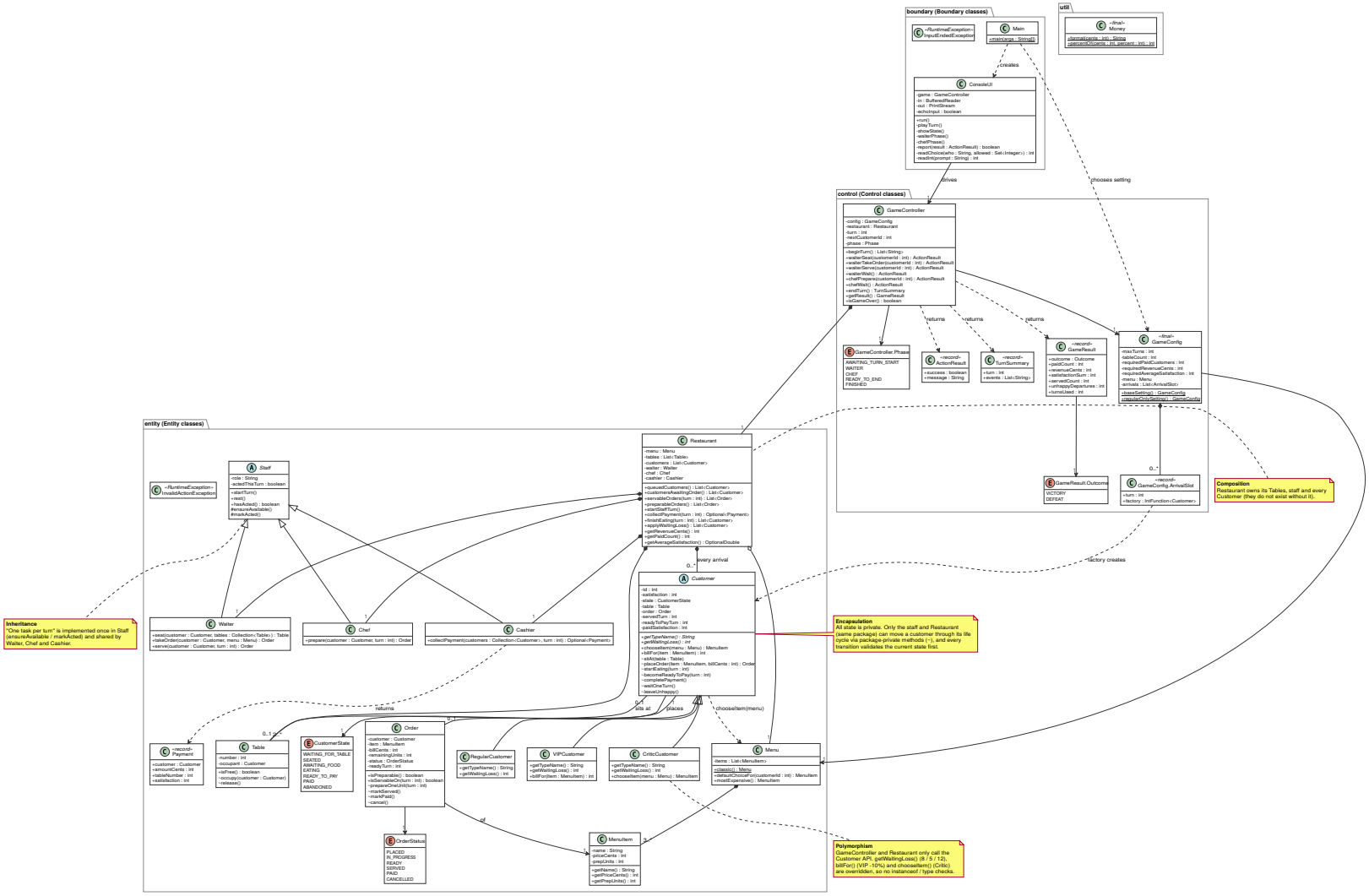


Figure B2. Class diagram at tag stage-2 (VIP and Critic customers). Vector graphic: zoom to read.